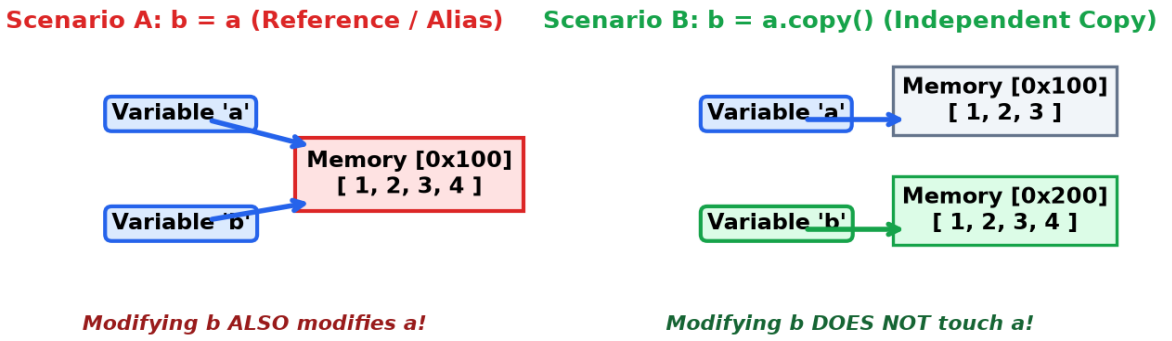
■ Data Science Master Study Notes & Architecture Reference

Student: **Amit** | Target: **Data Science & Practical Programming** | Course Version: **Comprehensive Textbook Edition**

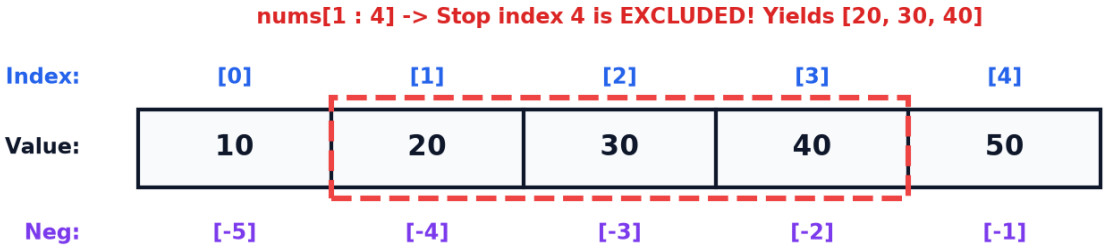
1. Python Memory Model: Reference Aliasing vs Copying

The Fundamental Law: Variables are reference labels (pointers). b = a does not allocate new RAM; both point to the same memory block (id(a) == id(b)). Mutating b silently alters a. Always use b = a.copy() to prevent corrupting raw datasets in Data Science.



2. Sequence Slicing Mechanics & The Half-Open Interval Rule

Formula: sequence[start : stop : step]. The interval is strictly **half-open**: [start, stop). The stop index is exclusive. Total elements = stop - start.



3. Safe Data Extraction: Dictionary .get() for Messy Datasets

```
customer = {"name": "Alice", "city": "Delhi"}
# Safe O(1) hash table lookup:
age = customer.get("age", 18)       # Missing -> fallback 18 without KeyError!
city = customer.get("city", "N/A")  # Exists  -> "Delhi"
```

4. Vectorized Thinking: List Comprehensions

```
prices = [120, 45, 300, 85, 520]
taxed_high_prices = [p * 1.10 for p in prices if p >= 100]  # [132.0, 330.0, 572.0]
```

5. Custom Data Cleaning Pipelines & Lambda Expressions

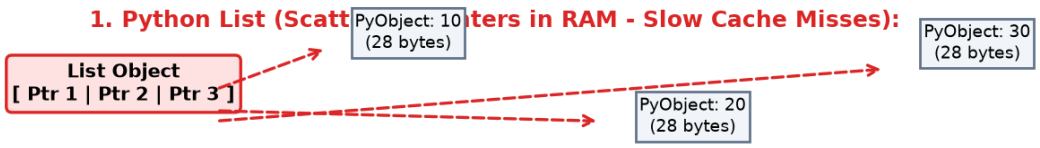
```
def clean_price(raw_val):
    if not isinstance(raw_val, str) or raw_val.strip().upper() in ["N/A", "MISSING", ""]:
        return 0.0
    return float(raw_val.replace("$", "").replace(",", "").strip())

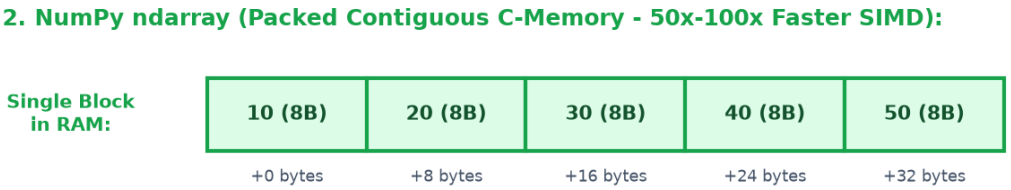
mph_to_kmh = lambda mph: round(mph * 1.60934, 2)
```

6. File Ingestion (CSV/JSON): The with open(...) as f Architecture

• open('file.csv', 'r'): Requests a live stream handle from Windows.
• as f: Assigns the live file handle to variable f.
• with: Context Manager that guarantees automatic f.close() exits.
• newline='': Prevents Windows carriage returns (\r\n) from injecting blank rows.
• csv.DictReader(f): Maps header rows to dictionary keys per line.
• json.load(f): Deserializes JSON feeds into native Python dicts and lists.

7. Phase 2: Numerical Computing with NumPy (Ndarrays & Vectorization)





NumPy stores numbers in a **contiguous block of raw C memory**, avoiding scattered pointer lookups. Every element must share the same dtype, enabling $O(1)$ address calculation: Address(i) = Base + (i * 8).

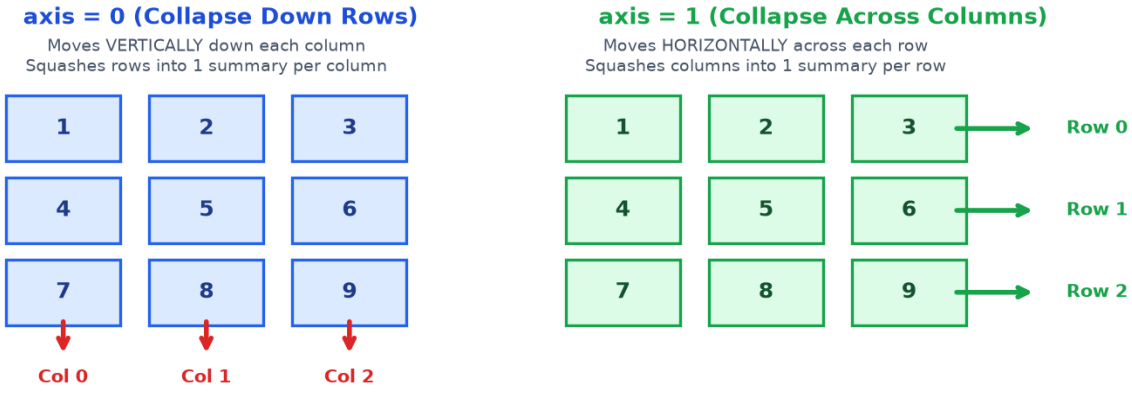
8. 2D Slicing, Matrix Axes (axis=0/1) & Boolean Masking

Multi-Dimensional Slicing: matrix[row_slice, col_slice]

Axis Collapse Rule: Specifying an axis tells NumPy which axis to **COLLAPSE**!

• axis=0: Moves **vertically** down rows, collapsing rows into 1 summary per column.

• axis=1: Moves **horizontally** across columns, collapsing columns into 1 summary per row.



9. Statistical Dispersion & Vectorized Anomaly Detection (Z-Scores)

In Data Science and Machine Learning, central tendency (mean, median) only tells half the story. **Dispersion** describes how spread out the data points are around that center.

• **Variance (σ^2):** The average of squared deviations: Var = $\Sigma (x - \mu)^2 / N$. Computed with np.var(a).
• **Standard Deviation (σ):** The square root of variance: Std = $\sqrt{Var}$. Computed with np.std(a).
• **The Z-Score Transformation:** Measures how many standard deviations a point is away from the mean:
$Z = (X - \mu) / \sigma$

Points with |Z| > 2.0 (or 3.0) are statistical outliers (e.g. server latency spikes or credit card fraud).

```
latencies = np.array([24.5, 26.1, 25.0, 24.8, 120.5, 25.4, 25.2, 26.0])
mean_val = np.mean(latencies)
std_val = np.std(latencies)

# Vectorized Z-Score:
z_scores = (latencies - mean_val) / std_val

# Extract raw anomaly values using boolean masking:
outliers = latencies[z_scores > 2.0]  # array([120.5])
```

10. Extrema Positioning (argmax/argmin) & Array Reshaping

10.1 Locating Positions with argmin & argmax:

While np.max() returns the maximum value, np.argmax() returns the **integer index** where that maximum occurred. Combined with axis=0 or axis=1, it answers vital business questions:

• np.argmax(sales, axis=1): Peak month for *each store* (row-wise collapse).
• np.argmax(sales, axis=0): Top performing store for *each month* (column-wise collapse).

```
# Reshaping with the -1 Wildcard:
arr = np.arange(10, 130, 10)         # 12 elements (1D)
grid = arr.reshape(3, 4)             # 2D matrix (3, 4)
grid_t = grid.T                      # Transpose -> shape (4, 3)

first_col = grid_t[:, 0]             # 1D array of 4 items: [10, 20, 30, 40]
col_vector = first_col.reshape(-1, 1) # 2D column vector of shape (4, 1)
```

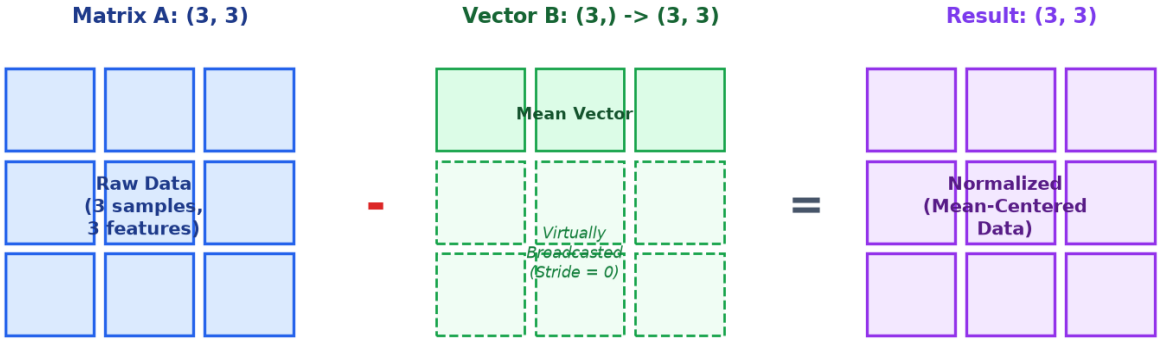
11. The NumPy Broadcasting Engine & Data Normalization

Broadcasting is NumPy's superpower. It allows arithmetic operations between arrays of **different shapes** without copying data in RAM. It creates **virtual dimensions** by setting the memory stride along that axis to **0**.

The Two Golden Rules of Broadcasting:

NumPy compares array shapes from **right to left** (trailing dimensions first):

1. Two dimensions are compatible if they are **equal**, OR
2. One of the dimensions is **1** (or missing, which gets prepended as 1).



Practical Machine Learning Application: Feature Mean-Centering

Subtracting the feature means ensures all features have zero mean, speeding up gradient descent:

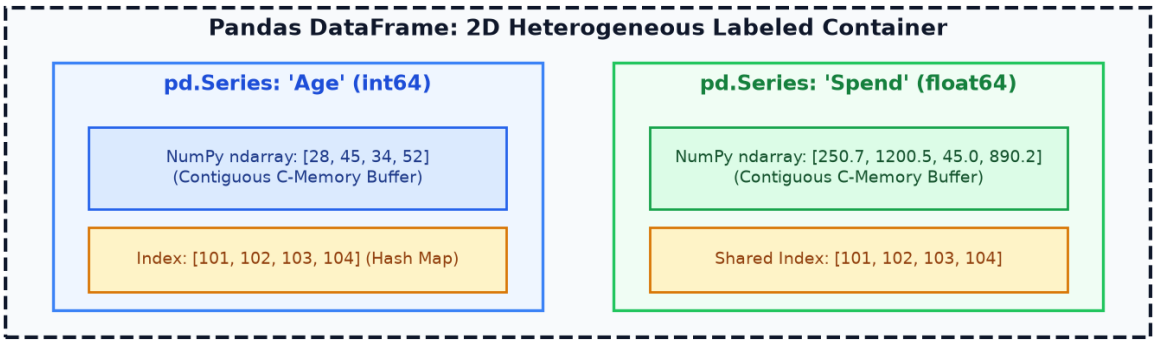
```
houses = np.array([
    [18.0, 3.0, 15.0],   # [Area, Bedrooms, Age]
    [24.0, 4.0,  5.0],
    [12.0, 2.0, 25.0]
])

# Feature means across all rows (collapses axis 0 -> shape (3,)):
feature_means = houses.mean(axis=0)  # [18.0, 3.0, 15.0]

# Vectorized Broadcasting: (3, 3) - (3,) -> Stretches across rows!
normalized_houses = houses - feature_means
```

12. Phase 3: Tabular Computing with Pandas Architecture

Pandas extends NumPy by providing **heterogeneous dtypes** and **semantic label indexing**. Under the hood, a pd.Series is a NumPy array paired with an Index hash-map, while a pd.DataFrame is a dictionary of Series sharing a common row Index.



12.1 Automatic Label-Based Alignment (The Pandas Superpower):

When performing arithmetic across two Series or DataFrames, Pandas matches rows by their **Index labels**, not their array positions. Missing keys naturally evaluate to NaN, preventing silent data misalignment. Use .add(fill_value=0) to safely aggregate disjoint datasets.

```
import pandas as pd

store_a = pd.Series([12000, 4500, 8000], index=["Laptops", "Mice", "Monitors"])
store_b = pd.Series([9500, 3200, 6000], index=["Laptops", "Mice", "Tablets"])

# Direct addition (+) creates NaNs for disjoint labels:
total_direct = store_a + store_b

# Safe addition fills missing labels with 0.0:
total_safe = store_a.add(store_b, fill_value=0)

# Vectorized Feature Engineering on DataFrames:
df["dti_ratio"] = df["monthly_debt"] / df["monthly_income"]
df["high_risk"] = df["dti_ratio"] > 0.40
```